

Algorithms

Personal Notes

Mihir Rao

Contents

1	Recursion	2
1.1	Reductions	2
1.2	Simplify and Delegate	2
1.3	Tower of Hanoi	3
1.4	Mergesort	4
1.5	Quicksort	5
1.6	The Divide-and-Conquer Pattern	6
1.7	Recursion Trees	6
1.8	Floors, Ceilings, and Domain Transformations	7
1.9	Linear-Time Selection	8
1.9.1	Median of Medians	8
1.10	Fast Multiplication	9
1.11	Exponentiation by Squaring	10

1 Recursion

Recursion is reduction applied to a problem itself. A recursive algorithm solves an instance directly when it reaches a base case; otherwise, it reduces the instance to one or more strictly simpler instances of the same problem, delegates those instances, and combines their solutions. Correctness is therefore proved by induction, while running time is described by a recurrence.

1.1 Reductions

Definition 1.1 (Reduction). A *reduction* from problem X to problem Y is an algorithm for X that uses a correct algorithm for Y as a black-box subroutine. Correctness of the reduction may depend on the specification of the subroutine, but not on its implementation.

This separation distinguishes correctness from efficiency. An algorithm that uses a priority queue can be correct for every valid priority-queue implementation, while its running time still depends on the data structure chosen to implement that interface. Replacing the implementation may improve performance without changing the outer algorithm or its proof.

***Note:** Algorithm design becomes more modular when a subroutine is treated as magic: state exactly what it promises, assume that promise, and ignore how it is fulfilled. The same abstraction lets a component be reused in settings its designer never anticipated.*

1.2 Simplify and Delegate

A recursive reduction follows two rules:

1. Solve elementary instances directly.
2. Reduce every other instance to strictly simpler instances of the same problem.

The second rule alone is insufficient. There must be no infinite descending chain of recursive calls. A well-founded size measure, such as a nonnegative integer that strictly decreases at every call, guarantees eventual arrival at a base case.

Proposition 1.2 (Recursive proof pattern). *Suppose an algorithm solves every base case correctly and, for every non-base instance of size n , combines correct solutions to strictly smaller instances into a correct solution of the original instance. Then the algorithm is correct for every valid input.*

Proof. Apply strong induction to the input size. Correctness on the smallest sizes is the base case. For an instance of size n , the induction hypothesis supplies correct answers for every smaller recursive subproblem; the combine step then produces the correct answer for size n . $\square$

Peasant multiplication illustrates the pattern through the identity

$$xy = \begin{cases} 0, & x = 0, \\ \lfloor x/2 \rfloor (2y), & x > 0 \text{ and } x \text{ is even,} \\ \lfloor x/2 \rfloor (2y) + y, & x \text{ is odd.} \end{cases}$$

The first argument decreases from x to $\lfloor x/2 \rfloor$, so recursion terminates at zero.

```

1 def peasant_multiply(x, y):
2     if x == 0:
3         return 0
4     product = peasant_multiply(x // 2, 2 * y)
5     return product + y if x % 2 else product

```

1.3 Tower of Hanoi

The Tower of Hanoi contains n disks ordered by size on a source peg. A legal move transfers the top disk of one peg to another without placing a larger disk above a smaller disk. To move the whole tower from source to destination using a temporary peg:

1. recursively move the top $n - 1$ disks to the temporary peg;
2. move disk n to the destination; and
3. recursively move the $n - 1$ disks from the temporary peg to the destination.

```

1 def hanoi(n, source, destination, temporary):
2     if n > 0:
3         hanoi(n - 1, source, temporary, destination)
4         move(n, source, destination)
5         hanoi(n - 1, temporary, destination, source)

```

The base case $n = 0$ correctly performs no moves. For $n > 0$, the induction hypothesis justifies both recursive transfers, and the only remaining move places the largest disk directly at its destination.

Theorem 1.3. *The recursive Hanoi algorithm uses exactly $2^n - 1$ moves on n disks.*

Proof. Let $T(n)$ be the number of moves. The algorithm gives

$$T(0) = 0, \quad T(n) = 2T(n-1) + 1.$$

Assuming $T(n-1) = 2^{n-1} - 1$,

$$T(n) = 2(2^{n-1} - 1) + 1 = 2^n - 1.$$

The claim follows by induction. □

This exponential growth is intrinsic to the procedure: moving the largest disk requires moving all smaller disks away first and restoring them afterward.

1.4 Mergesort

Mergesort performs its easy work before recursion and its substantial work afterward:

1. divide the array into two nearly equal subarrays;
2. recursively sort both subarrays; and
3. merge the sorted subarrays.

```
1 def merge_sort(a):
2     if len(a) <= 1:
3         return a
4     middle = len(a) // 2
5     left = merge_sort(a[:middle])
6     right = merge_sort(a[middle:])
7     return merge(left, right)
```

The merge procedure repeatedly removes the smaller first element of the two sorted sequences. If one sequence is empty, it appends the remainder of the other.

Lemma 1.4. *Given two sorted sequences with total length n , MERGE returns their elements in sorted order in $O(n)$ time.*

Proof. Maintain the invariant that the output contains exactly the smallest elements already removed from the inputs, in sorted order. At each step, the next smallest remaining element must be at the front of one sequence, so choosing the smaller front element preserves the invariant. Once either input is empty, the other is already sorted and can be appended. Each element is processed once. $\square$

Theorem 1.5. *Mergesort correctly sorts any array of n elements in $O(n \log n)$ time.*

Proof. Correctness follows by induction on n . Arrays of length at most one are already sorted. The induction hypothesis sorts both smaller halves, and the lemma merges them correctly. For running time, the two recursive calls and linear merge step give

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) = 2T(n/2) + O(n).$$

Every level of the recursion tree performs $O(n)$ work, and the tree has $O(\log n)$ levels. $\square$

1.5 Quicksort

Quicksort reverses the distribution of work:

1. choose a pivot;
2. partition the array into elements smaller than the pivot, the pivot, and elements at least as large as the pivot; and
3. recursively sort the two outer subarrays.

```
1 def quick_sort(a):
2     if len(a) <= 1:
3         return a
4     pivot = choose_pivot(a)
5     smaller, equal, larger = partition(a, pivot)
6     return quick_sort(smaller) + equal + quick_sort(larger)
```

An in-place partition maintains a boundary ℓ such that processed entries at indices at most ℓ are smaller than the pivot and later processed entries are not. Swapping each newly discovered smaller entry across that boundary preserves the invariant.

Proposition 1.6. *Quicksort is correct for every choice of pivot.*

Proof. Induct on the array length. Partition places every smaller element before the pivot and every larger element after it. The induction hypothesis correctly sorts both strictly smaller subarrays, and their concatenation around the pivot is therefore sorted. $\square$

If the pivot has rank r , the running time satisfies

$$T(n) = T(r - 1) + T(n - r) + O(n).$$

A median pivot gives two balanced subproblems and $T(n) = O(n \log n)$. A consistently extreme pivot gives

$$T(n) = T(n - 1) + O(n) = O(n^2).$$

Choosing the median of the first, middle, and last elements often helps in practice, but it does not improve the quadratic worst-case bound. Randomizing the pivot makes the algorithm's guarantee depend on randomness controlled by the algorithm rather than on an assumed distribution of inputs.

1.6 The Divide-and-Conquer Pattern

Definition 1.7 (Divide and conquer). A *divide-and-conquer* algorithm:

1. divides an instance into independent smaller instances of the same problem;
2. recursively solves those instances; and
3. combines their solutions.

Below a fixed size threshold, it solves the instance directly.

Mergesort performs almost no work while dividing and linear work while combining. Quicksort performs linear work while dividing and almost no work while combining. Both are manifestations of the same recursive structure.

1.7 Recursion Trees

Definition 1.8 (Recursion tree). A *recursion tree* has one node for each recursive subproblem. Each node is labeled by the nonrecursive work performed on that subproblem. The total running time is the sum of all node labels.

Consider

$$T(n) = rT(n/c) + f(n),$$

where a size- n problem makes r recursive calls of size n/c and performs $f(n)$ additional work. At depth i , the tree has r^i nodes of size n/c^i , so the total contribution of that level is

$$r^i f(n/c^i).$$

With a constant-size base case, the depth is $L = \log_c n$, and

$$T(n) = \sum_{i=0}^L r^i f(n/c^i).$$

There are $r^L = n^{\log_c r}$ leaves.

Three geometric patterns occur repeatedly:

Level totals	Dominant contribution	Result
Exponentially decreasing	root	$T(n) = O(f(n))$
Equal	all $L + 1$ levels	$T(n) = O(f(n) \log n)$
Exponentially increasing	leaves	$T(n) = O(n^{\log_c r})$

For mergesort, $r = 2$, $c = 2$, and $f(n) = n$, so every level contributes $\Theta(n)$ and $T(n) = O(n \log n)$. Balanced subproblems are not required:

$$T(n) \leq T(n/3) + T(2n/3) + O(n)$$

also has $O(n)$ work per level and $O(\log n)$ depth. In contrast, the worst-case quicksort tree has $O(n)$ depth and at most $O(n)$ work per level, giving $O(n^2)$.

1.8 Floors, Ceilings, and Domain Transformations

Input sizes are integers, so exact recurrences often contain floors and ceilings. For an asymptotic upper bound, these terms can usually be removed by a *domain transformation*. For mergesort,

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) \leq 2T(n/2 + 1) + O(n).$$

Define $S(n) = T(n + 2)$. Then

$$S(n) \leq 2S(n/2) + O(n),$$

whose recursion tree gives $S(n) = O(n \log n)$. Therefore $T(n) = S(n - 2) = O(n \log n)$. Similar constant shifts remove floors, ceilings, and lower-order terms from many divide-and-conquer recurrences.

1.9 Linear-Time Selection

The selection problem asks for the k th smallest element of an unsorted array. Quickselect partitions around a pivot exactly as quicksort does, but recurses only into the side containing the requested rank.

```
1 def quick_select(a, k):
2     if len(a) == 1:
3         return a[0]
4     pivot = choose_pivot(a)
5     smaller, equal, larger = partition(a, pivot)
6     if k <= len(smaller):
7         return quick_select(smaller, k)
8     if k <= len(smaller) + len(equal):
9         return pivot
10    return quick_select(larger, k - len(smaller) - len(equal))
```

Correctness does not depend on pivot quality, but running time does. An extreme pivot gives $T(n) = T(n - 1) + O(n) = O(n^2)$. Any pivot that leaves at most αn elements on the recursive side for a fixed $\alpha < 1$ gives

$$T(n) \leq T(\alpha n) + O(n) = O(n),$$

because the recursion-tree level totals decrease geometrically.

1.9.1 Median of Medians

The Blum-Floyd-Pratt-Rivest-Tarjan algorithm deterministically constructs a good pivot:

1. divide the array into groups of five;
2. find each group median by brute force;
3. recursively select the median of those medians; and
4. use that value as the quickselect pivot.

At least half of the group medians are no larger than the median of medians, and each such median is no smaller than two other entries in its group. Therefore at least approximately $3n/10$ entries lie on each side of the pivot, so the selected recursive side contains at most $7n/10$ entries.

Theorem 1.9. *Median-of-medians selection runs in $O(n)$ worst-case time.*

Proof. Selecting the pivot recursively uses an array of size $n/5$; searching after partitioning uses at most $7n/10$ entries. All grouping and partitioning work is linear, so

$$T(n) \leq T(n/5) + T(7n/10) + O(n).$$

The total subproblem size at the next level is at most

$$\frac{n}{5} + \frac{7n}{10} = \frac{9n}{10}.$$

Thus successive recursion-tree levels form a decreasing geometric series:

$$O(n) \sum_{i \geq 0} \left(\frac{9}{10}\right)^i = O(n).$$

□

Groups of three would yield

$$T(n) \leq T(n/3) + T(2n/3) + O(n) = O(n \log n),$$

so five is the smallest useful odd group size for this analysis.

1.10 Fast Multiplication

Split two n -digit numbers at $m = \lceil n/2 \rceil$:

$$x = 10^m a + b, \quad y = 10^m c + d.$$

Direct expansion gives

$$xy = 10^{2m} ac + 10^m(ad + bc) + bd.$$

Computing all four products recursively yields

$$T(n) = 4T(n/2) + O(n) = O(n^2),$$

which does not improve on grade-school multiplication.

Karatsuba's identity computes the middle coefficient with only one additional product:

$$ad + bc = ac + bd - (a - b)(c - d).$$

The algorithm recursively computes ac , bd , and $(a - b)(c - d)$, then combines them using linear-time

additions and digit shifts.

Theorem 1.10 (Karatsuba multiplication). *Two n -digit numbers can be multiplied in $O(n^{\log_2 3}) = O(n^{1.585})$ time.*

Proof. Three half-size recursive products and linear nonrecursive work give

$$T(n) = 3T(n/2) + O(n).$$

The recursion-tree level totals increase geometrically and are dominated by the $n^{\log_2 3}$ leaves. $\square$

More elaborate splitting schemes lead to Toom-Cook multiplication and, eventually, FFT-based algorithms with nearly linear running time.

1.11 Exponentiation by Squaring

Repeatedly multiplying by a computes a^n using $n - 1$ multiplications. The recursive identity

$$a^n = \begin{cases} 1, & n = 0, \\ (a^{n/2})^2, & n > 0 \text{ and } n \text{ is even,} \\ (a^{\lfloor n/2 \rfloor})^2 a, & n \text{ is odd} \end{cases}$$

reduces the exponent by roughly half at every step.

```
1 def power(a, n):
2     if n == 0:
3         return 1
4     half = power(a, n // 2)
5     square = half * half
6     return square if n % 2 == 0 else square * a
```

The number of multiplications satisfies

$$T(n) \leq T(\lfloor n/2 \rfloor) + 2 = O(\log n).$$

This bound is asymptotically optimal: after k multiplications, the largest power that can have been constructed is at most 2^k , so computing a^n requires $\Omega(\log n)$ multiplications.

Note: *The method applies to any associative multiplication with an identity, not only real numbers. Important examples include modular arithmetic and matrix powers. For particular exponents, shorter addition chains may use fewer multiplications, but logarithmic growth is unavoidable.*

Source. These notes synthesize Chapter 1, “Recursion,” of Jeff Erickson’s *Algorithms* (2019), available under the Creative Commons Attribution 4.0 International License at the book website.